Sorting Algorithms

Experimental Analysis

A Comprehensive Experimental Comparison of Classical and Modern Sorting Algorithms

Author: Claudia Lara Carvalho
Date: June 2026
Repository: https://github.com/qlaudialara/sorting-algorithms

Table of Contents

1. Executive Summary

This project provides a comprehensive experimental comparison of 10 classical and modern sorting algorithms implemented in Python 3.13. The study conducts over 6,000 individual experimental runs to investigate whether the practical behaviour of sorting algorithms matches their theoretical complexity under various conditions. The framework generates publication-quality visualizations and detailed statistical analysis for academic research purposes.

2. Project Overview

Objective: Provide empirical evaluation of sorting algorithms and explore how factors such as input size, data type, and initial ordering influence performance.

Scope: 10 sorting algorithms, 3 data types, 5 input structures, 4 input size ranges, 10 runs per configuration.

Total Configurations: ~6,000+ individual experimental measurements with comprehensive statistical analysis.

3. Algorithms Implemented

Algorithm	Complexity	Type	Notes
Bubble Sort	$O(n^2)$	Quadratic	Simple, poor scalability
Insertion Sort	$O(n^2)$	Quadratic	Good for nearly sorted data
Selection Sort	$O(n^2)$	Quadratic	Consistent performance
Merge Sort	O(n log n)	Efficient	Guaranteed, stable, O(n) space
Quick Sort	O(n log n) avg	Efficient	Pivot strategy analysis included
Heap Sort	O(n log n)	Efficient	In-place, no extra space
Counting Sort	O(n + k)	Specialized	Integers only
Radix Sort	O(d × (n+k))	Specialized	Non-negative integers only
Tim Sort	O(n log n)	Efficient	Hybrid, custom implementation
sorted()	O(n log n)	Reference	Python's built-in implementation

4. Experimental Design

Input Sizes:

• Quadratic algorithms ($O(n^2)$): 1,000 | 5,000 | 10,000 elements

• Efficient algorithms (O(n log n)): 1,000 | 10,000 | 50,000 | 100,000 elements

Data Types:

• Integers: Random values in [0, 1,000,000]

• Floats: Random values in [0.0, 1,000.0]

• Strings: Random alphanumeric strings (length 10)

Input Structures:

• Random: Completely random arrangement

• Sorted: Pre-sorted in ascending order

• Reversed: Pre-sorted in descending order

• Nearly Sorted: 95% sorted with 5% random perturbations

• Flat: All elements identical

Statistical Rigor:

• 10 runs per configuration for statistical significance

• High-precision timing using time.perf_counter()

• Statistical metrics: mean, std dev, min, max execution times

5. Project Structure

The project is organized into modular components for clean separation of concerns and code reusability:

Module	Lines	Purpose
algorithms/implementations.py	393	10 sorting algorithm implementations
data_generation/__init__.py	156	Dataset generation utilities
experiments/__init__.py	242	Experiment runner and timing
visualization/__init__.py	298	Publication-quality plotting
main.py	234	Main orchestration script
quick_test.py	91	Quick validation test
TOTAL	1,439	Total lines of production code

6. Technologies Used

• **Python 3.13:** Core language for implementation

• **Matplotlib:** Visualization and plotting

• **Pandas:** Data handling and CSV export

• **NumPy:** Numerical computations

• **Seaborn:** Statistical visualization enhancement

7. Installation & Usage

Clone Repository:
git clone https://github.com/qlaudialara/sorting-algorithms.git
cd sorting-algorithms

Install Dependencies:
pip install -r requirements.txt

Quick Test (2-3 minutes):
python3 quick_test.py

Full Experiment (30-60 minutes):
python3 main.py

8. Expected Results

Upon completion, the project generates comprehensive results in *outputs/experiment_YYYYMMDD_HHMMSS/*:

- **experiment_results.csv:** Complete dataset with all measurements
- **01_overall_comparison.png:** All algorithms performance overview
- **02_by_datatype_*.png:** Performance by data type (3 plots)
- **03_by_structure_*.png:** Performance by input structure (5 plots)
- **04_quicksort_worst_case.png:** Quick Sort pivot strategy analysis
- **05_timsort_vs_builtin.png:** Custom vs Python's built-in Tim Sort
- **06_quadratic_vs_efficient.png:** Complexity class comparison
- **Console rankings:** Algorithm rankings by category

9. Key Features

✓ **Modular Design** – Clean separation of concerns

✓ **Comprehensive Coverage** – 10 algorithms × 3 types × 5 structures

✓ **Publication Ready** – High-DPI figures with proper labels

✓ **Reproducible** – Fixed random seeds for consistency

✓ **Scalable** – Easy to add algorithms or modify parameters

✓ **Statistical Rigor** – Multiple runs with comprehensive metrics

✓ **Well Documented** – Type hints, docstrings, comments

✓ **Professional** – Suitable for academic research

10. References

• Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

• Skiena, S. S. (2008). The Algorithm Design Manual (2nd ed.). Springer.

• Python Software Foundation. (2024). Python Documentation – Sorting.

• Peters, T. (2002). Timsort Description and Analysis.

Project Repository: https://github.com/qlaudialara/sorting-algorithms
Author: Claudia Lara Carvalho
Report Generated: June 22, 2026 at 16:00:10
Status: Production Ready